Agentes y datos: banco de actividades

1. Qué problema resuelve el sistema

Cuando un docente falta, el curso necesita una actividad lista para trabajar. Este sistema lo resuelve con un **banco de actividades**.

- Los docentes preparan y aprueban actividades **con anticipación**, sin saber cuándo van a faltar.
- Las actividades aprobadas quedan guardadas en el banco.
- Cuando un docente falta, coordinación elige una actividad del banco y la **activa** para una comisión y una fecha.
- Los estudiantes resuelven la actividad, la entregan y el sistema la corrige.

Una misma actividad del banco puede activarse muchas veces, en distintas fechas.

En esta clase no vamos a programar. Vamos a entender qué hace cada parte del sistema y a decidir **qué datos necesita guardar cada una**.

2. Componentes, agentes y módulos

El sistema se divide en **cinco componentes**. Cada componente es una parte del sistema con una sola tarea, y cada equipo trabaja con uno.

Todo componente funciona igual:

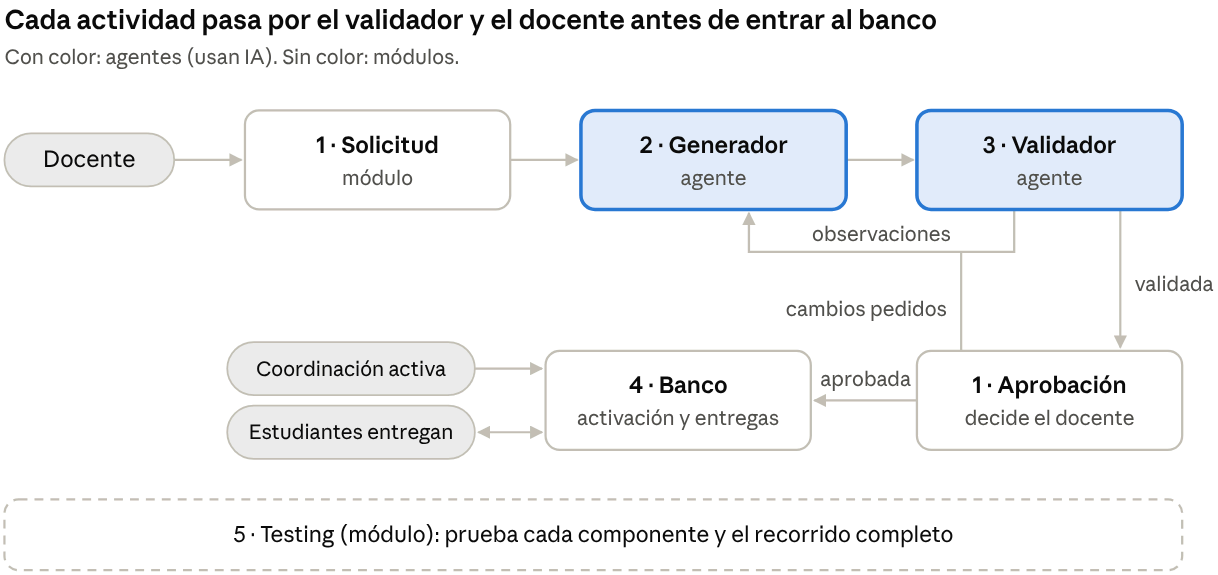
1. **Entrada:** la información que recibe.
2. **Reglas:** lo que tiene que respetar o controlar mientras trabaja.
3. **Salida:** lo que produce y deja disponible para otro componente.

Hay dos tipos de componente:

Tipo	Qué lo distingue	Componentes
Agente	Usa un modelo de inteligencia artificial para producir su salida.	2 y 3
Módulo	Sigue reglas fijas escritas en código. No usa inteligencia artificial.	1, 4 y 5

Ejemplo cotidiano. En un kiosco, el cajero es un componente. Entrada: los productos y el dinero. Reglas: el precio de cada producto y no vender lo que no tiene stock. Salida: el ticket y el vuelto. El ticket queda guardado porque el dueño lo necesita después para hacer la caja.

3. Cómo se conectan



cómo se conectan los cinco componentes

La salida de un componente es la entrada del siguiente. Si el validador o el docente encuentran un problema, la actividad vuelve al generador con observaciones. Testing no forma parte del recorrido: prueba a todos los demás.

4. Ficha de cada componente

Componente 1 · Solicitud y aprobación (módulo)

Misión: el docente pide una actividad y, más tarde, decide si la aprueba.

Entrada

- Lo que carga el docente: materia, comisión, tema, objetivo, cantidad de preguntas, duración y material permitido (por ejemplo, un PDF de la clase).
- Más tarde: la actividad que ya pasó por el validador.

Reglas

- No acepta una solicitud con campos obligatorios vacíos.
- Solo el docente de esa materia puede aprobar la actividad.
- Si el docente pide cambios, tiene que escribir qué cambiar.

Salida

- Una solicitud completa, lista para el generador.
- La decisión del docente: aprobada, o cambios pedidos con sus observaciones.

Componente 2 · Generador (agente)

Misión: crear la actividad con inteligencia artificial a partir de la solicitud.

Entrada

- La solicitud completa.
- El material permitido.
- Las instrucciones que se le dan al modelo de IA.
- Si la actividad vuelve: las observaciones del validador o del docente.

Reglas

- Usa solo el material permitido.
- Genera la cantidad de preguntas pedida, todas cerradas, con una sola opción correcta.
- La opción correcta no queda siempre en la misma posición.
- Entrega siempre con el mismo formato, para que otro componente pueda leerlo.

Salida

- Un borrador de actividad: consigna, preguntas, opciones y respuesta correcta de cada pregunta.

Componente 3 · Validador (agente)

Misión: revisar si el borrador cumple lo que pidió el docente antes de que el docente lo vea.

Entrada

- El borrador.
- La solicitud original, para comparar.

Reglas

- Controles con código: formato correcto, cantidad de preguntas, ningún campo vacío.
- Controles con IA: si respeta el tema, la dificultad y el material permitido.
- La IA puede equivocarse: este control no reemplaza la aprobación del docente.

Salida

- Borrador validado, que pasa al docente, o
- Lista de observaciones, que vuelve al generador.

Componente 4 · Banco, activación y entregas (módulo)

Misión: guardar las actividades aprobadas, permitir que coordinación las active y recibir y corregir las entregas.

Entrada

- La actividad aprobada.
- El pedido de coordinación: qué actividad, para qué comisión y qué fecha.
- Las respuestas de cada estudiante.

Reglas

- Solo entran al banco actividades aprobadas por el docente.
- Una actividad puede activarse muchas veces.
- Solo entregan los estudiantes de la comisión activada, una entrega por estudiante.
- Corrige con código, comparando cada respuesta con la correcta.
- El estudiante nunca ve la respuesta correcta antes de entregar.
- Los estudiantes son menores: se pide solo la información personal imprescindible.

Salida

- La actividad disponible en el banco.
- La activación: actividad, comisión, fecha y quién la activó.
- Cada entrega con sus respuestas y su resultado.

Componente 5 · Testing (módulo)

Misión: comprobar que cada componente cumple sus reglas y que el recorrido completo funciona.

Entrada

- Las reglas de los componentes 1 a 4.

- Los casos de prueba: qué se prueba, con qué datos y qué resultado se espera.

Reglas

- Prueba cada componente por separado y después todos juntos.
- Nunca modifica datos reales: usa datos de prueba.
- Cada falla se registra con su evidencia.

Salida

- El resultado de cada prueba y la lista de fallas encontradas.

5. Ejemplo: una actividad, dos ausencias

Una docente de Introducción a Testing prepara en octubre una actividad sobre métodos HTTP. No sabe si va a faltar: la deja lista por si acaso.

1. **Solicitud.** Completa el formulario: tema «Métodos HTTP», 4 preguntas, 40 minutos, y sube el PDF de la clase de APIs como material permitido.
2. **Generador.** Crea un borrador con 4 preguntas usando el PDF.
3. **Validador.** Detecta que una pregunta trata un tema que no está en el PDF. El borrador vuelve al generador con esa observación.
4. **Generador.** Crea un segundo borrador sin esa pregunta. El validador ahora lo acepta.
5. **Aprobación.** La docente revisa el segundo borrador y lo aprueba. La actividad entra al banco.
6. **Primera ausencia.** Semanas después, la docente falta. Coordinación busca en el banco, elige esta actividad y la activa para 5.º John Backus en esa fecha.
7. **Entregas.** Cada estudiante responde y entrega. El sistema corrige y registra el resultado de cada uno.
8. **Segunda ausencia.** Otro día, coordinación vuelve a activar la misma actividad. Es la misma actividad, pero otra activación, con sus propias entregas.

Mientras tanto, el componente de testing prueba cada paso con datos de prueba.

6. De la salida al dato: dónde se guarda cada cosa

Cada salida de un componente es información. Para cada una hay que decidir si se guarda y dónde. Hay cuatro opciones:

Destino	Qué se guarda ahí	Ejemplo en este sistema
Base de datos	Datos con estructura, que se buscan, se relacionan y se actualizan.	Una solicitud, una entrega
Storage	Archivos completos.	El PDF de la clase
GitHub	El código y las instrucciones del sistema, no los datos de uso.	Las instrucciones que se le dan al modelo de IA
No se guarda	Lo que se usa durante un paso y después no se necesita.	La lista de actividades que ve coordinación mientras busca

Para decidir, sirven dos preguntas:

- **¿Alguien lo necesita después?** Otro componente, el docente, coordinación o el mismo componente más tarde.
- **¿Qué se pierde si no está?** Si la respuesta es «nada», probablemente no hay que guardarlo.

Guardar de más también es un problema: datos repetidos que se contradicen, información personal sin motivo y una base difícil de mantener.

7. Consigna del equipo

Cada equipo trabaja con su componente y completa la tabla con **todos** los datos que el componente recibe o produce. La primera fila es un ejemplo del componente 1.

Dato	De dónde viene	Quién lo usa después	Dónde se guarda	Por qué
Tema de la solicitud	Lo carga el docente	Generador y validador	Base de datos	El validador lo compara con el borrador

Después, responder con ejemplos del propio componente:

1. ¿Qué dato de nuestra tabla también aparece en la tabla de otro componente?
2. ¿Qué dejaría de funcionar si no guardáramos nada?

3. ¿Hay algo que necesitemos guardar más de una vez, por ejemplo cada vez que cambia?
4. ¿Cómo sabe otro componente que nuestro trabajo ya está terminado?
5. ¿Qué información personal manejamos y cuál es realmente necesaria?

8. Puesta en común

Los cinco componentes usan **una sola base de datos**. Por eso, al final, juntamos las cinco tablas y las convertimos en un único diseño.

Entre todos respondemos:

1. ¿Qué datos aparecen en más de una tabla? ¿Se guardan una vez o varias?
2. ¿Cómo sabe cada componente en qué etapa está una actividad?
3. ¿Qué diferencia hay entre una actividad y una activación? ¿Qué datos son de cada una?
4. Si un borrador vuelve dos veces al generador, ¿qué se conserva de las versiones anteriores?
5. ¿Qué dato une una entrega con la activación a la que pertenece?

Resultado de la clase: la lista de datos de todo el sistema, sin repeticiones, y para cada dato, dónde se guarda y por qué.